

ITU-MiniTwit

KSDSESM1KU — DevOps, Software Evolution and Software
Maintenance

Leo Sakharoff — leos@itu.dk

Peter Juul Møller — pemoe@itu.dk

Peter Kvist — peht@itu.dk

Apoorva Sood — apso@itu.dk

Håkon Refsvik — s25129@itu.dk

May 18, 2026

1 System's Perspective

1.1 Design and Architecture

Author(s): Peter K, Håkon

The system runs on a three-node Docker Swarm cluster hosted on DigitalOcean. The manager node handles orchestration and hosts the monitoring stack (Prometheus, Grafana, Loki) and Traefik, which terminates TLS and routes incoming traffic. All three nodes run a Webserver replica and a Promtail instance for log collection. PostgreSQL runs as a DigitalOcean managed instance, reachable from all swarm nodes over TCP on port 25060.

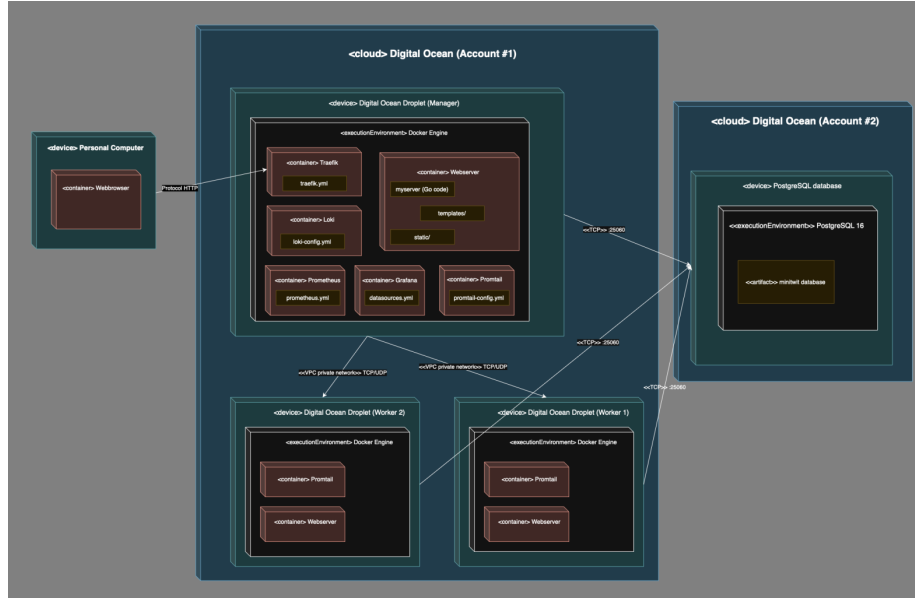


Figure 1: Deployment diagram showing the three-node Swarm cluster and external PostgreSQL database

At runtime, Traefik distributes HTTPS traffic from browsers and HTTP traffic from the simulator across the three Webserver replicas. Each replica connects to the shared PostgreSQL database over TCP/SSL. Prometheus scrapes metrics from all replicas via `/metrics`. Promtail runs as a global Swarm service (one per node), reads container logs from the Docker socket, and ships them to Loki. Grafana queries both Prometheus and Loki, and routes alerts to Discord via a webhook.

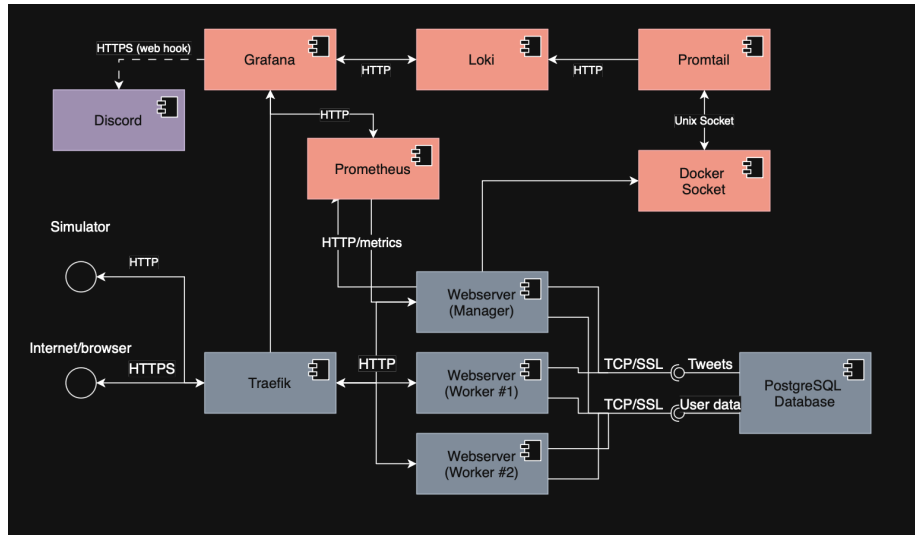


Figure 2: Component-and-connector diagram showing runtime components and their communication

1.2 Dependencies

Author(s): Peter K

1.2.1 Language & Framework

- **Go 1.25** — backend language
- **Gorilla Mux** — Matches incoming HTTP requests to handler functions (eg. /login)
- **GORM** — ORM for database access
- **Gorilla Sessions** — session management

1.2.2 Database

- **PostgreSQL** — primary database (accessed via GORM)

1.2.3 Infrastructure

- **Docker & Docker Swarm** — containerisation and orchestration
- **DigitalOcean** — cloud hosting
- **Terraform** — infrastructure as code
- **Traefik v3.6** — reverse proxy and automatic TLS

1.2.4 CI/CD

- **GitHub Actions** — automated build, test, and deploy pipeline

1.2.5 Observability

- **Prometheus** — metrics collection
- **Grafana** — dashboards and visualisation
- **Loki + Promtail** — log aggregation

1.2.6 Code Quality & Security

- **golangci-lint** — static analysis for Go
- **Hadolint** — Dockerfile linting
- **Semgrep** — static security analysis
- **Docker Scout** — container vulnerability scanning

1.3 Current State

Author(s): Peter J

We run four static analysis tools in CI on every pull request, all blocking merges if they fail.

golangci-lint is our main Go linter. We have **errcheck**, **govet**, **staticcheck**, **ineffassign**, and **unused** enabled. **errcheck** catches unchecked errors in Go code, **govet** catches common mistakes like misused format strings, **staticcheck** does deeper analysis for bugs and dead code, and **ineffassign** and **unused** flag variables or code that serve no purpose.

hadolint lints the Dockerfile against a set of best practice rules — things like pinning base image versions, avoiding **apt-get upgrade**, and making sure **RUN** layers are structured cleanly.

Semgrep does static application security testing (SAST) using three rulesets: **p/security-audit** for general security issues, **p/secrets** for accidentally committed credentials, and **p/owasp-top-ten** for common web vulnerabilities like SQL injection and XSS.

Docker Scout scans the final built Docker image for known CVEs in the OS packages and dependencies. It's configured to fail the build if any critical or high severity vulnerability is found, so we catch issues in base image dependencies before they reach production.

2 Process Perspective

2.1 CI/CD Pipeline

Author(s): Peter K, Håkon & Apoorva

The pipeline has two workflows: **CI** runs on every pull request to **master**; **CD** runs on every merge into **master**.

CI runs four jobs. **lint** runs first — **gofmt**, **golangci-lint**, and **hadolint**. **semgrep** and **test** run in parallel after **lint**. **semgrep** does SAST across three rulesets; **test** builds the image and runs unit tests, API tests, and Playwright browser tests inside Docker Compose. **docker-scout** runs last and scans the final image for CVEs, failing on any critical or high severity finding.

CD builds and pushes three images to GHCR (**minitwit**, **minitwit-prometheus**, **minitwit-grafana**), then SSHes into the Swarm manager and runs **docker stack deploy --with-registry-auth**. The flag passes registry credentials from the manager to worker nodes so they can pull from the private registry. We use a long-lived PAT for this rather than the ephemeral **GITHUB_TOKEN** — workers schedule pulls asynchronously, and the short-lived token had expired by the time workers pulled, causing “No such image” failures (incident: 21 Apr 2026).

Swarm then rolls out the update one replica at a time (**order: start-first**), so the new replica passes its health check before the old one is taken down.

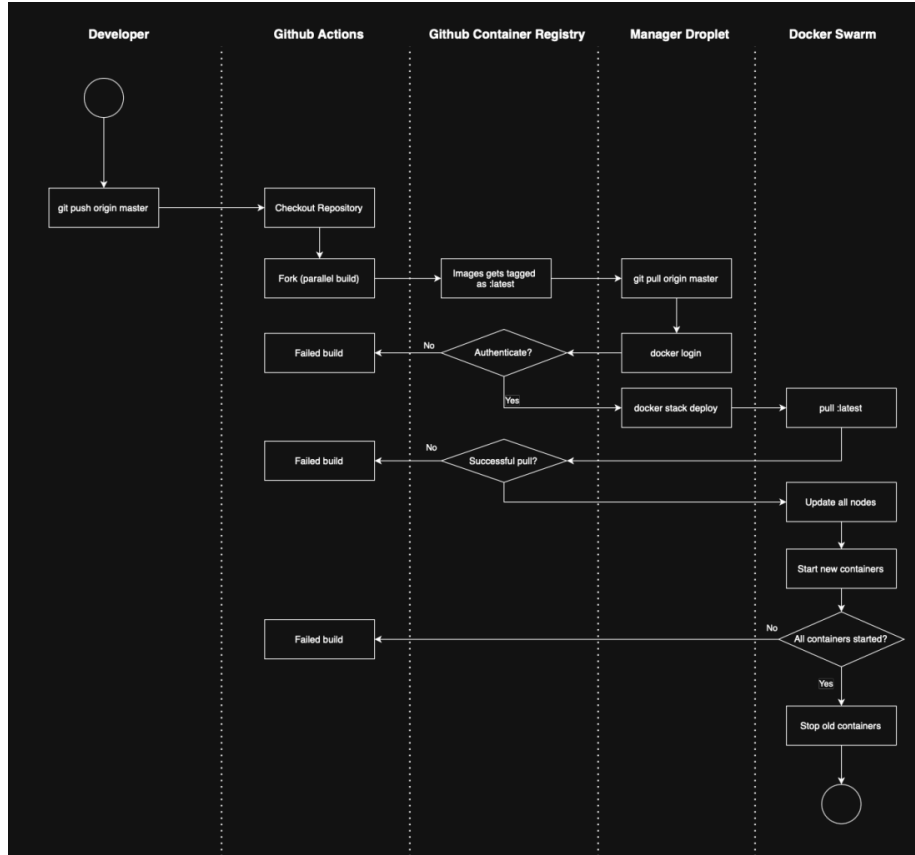


Figure 3: CD pipeline activity diagram: from git push to running containers in the Swarm

Browser tests are split out from API tests so failures became easier to diagnose, and the workflow was corrected when an invalid severity input caused the analysis step to misbehave. This keeps the quality gates useful instead of noisy.

Codacy was added late enough that some maintainability issues were only diagnosed after the fact, but it still provided a useful backstop during the final stages of the project. The pipeline was also exercised manually whenever the **YAML** changed, especially when **Playwright** was introduced, so the team could confirm the workflow still ran the expected jobs and that **Docker Scout** continued to gate only critical and high vulnerabilities.

2.2 Monitoring

Author(s): Peter J, Apoorva

For monitoring we use Prometheus to scrape metrics from the app, and

Grafana to visualize them. Inside the Go application we added a middleware that wraps every HTTP handler and records two custom metrics: `minitwit_http_responses_total` (a counter per route, method, and status code) and `minitwit_http_duration_seconds` (a latency histogram). These get exposed on `/metrics` and Prometheus scrapes all three webserver replicas every 15 seconds using DNS-based service discovery on the Swarm overlay network — so adding or removing replicas requires no config changes.

We set up three alert rules in Prometheus: one that fires if a webserver replica has been unreachable for over a minute, one for when more than 10% of responses are 5xx errors over a five-minute window, and one for when P95 latency goes above 1 second for five minutes. All alerts are routed to a Discord channel through a Grafana contact point, using a webhook URL stored as a Docker Swarm secret so it never ends up in the codebase.

DigitalOcean alert policies were also applied directly to the managed PostgreSQL instance. The database monitors disk usage, CPU usage, and memory usage, and the CPU policy in particular uses a 90% threshold over five minutes. When that limit is reached, the database emits the familiar “CPU is running high” warning for `db-postgresql-fra1-53911`, which gives the project early notice that the managed database is under pressure.

The Grafana setup is fully provisioned from code — datasources and dashboards are baked into a custom Docker image. The main dashboard covers uptime and availability across all replicas, total HTTP responses broken down by route, P95 response time, and a combined error rate panel. Over the week of 9–15 May we saw 100% uptime, and P95 response times stayed comfortably under 50 ms for the vast majority of the time.

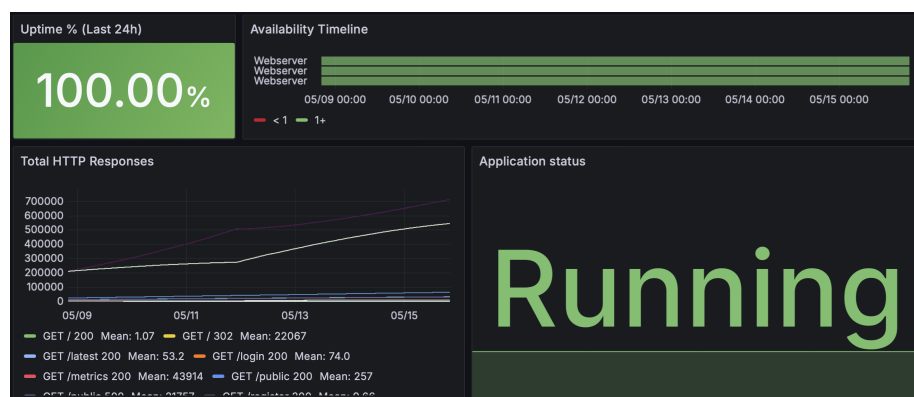


Figure 4: Availability dashboard showing 100% uptime and HTTP response breakdown

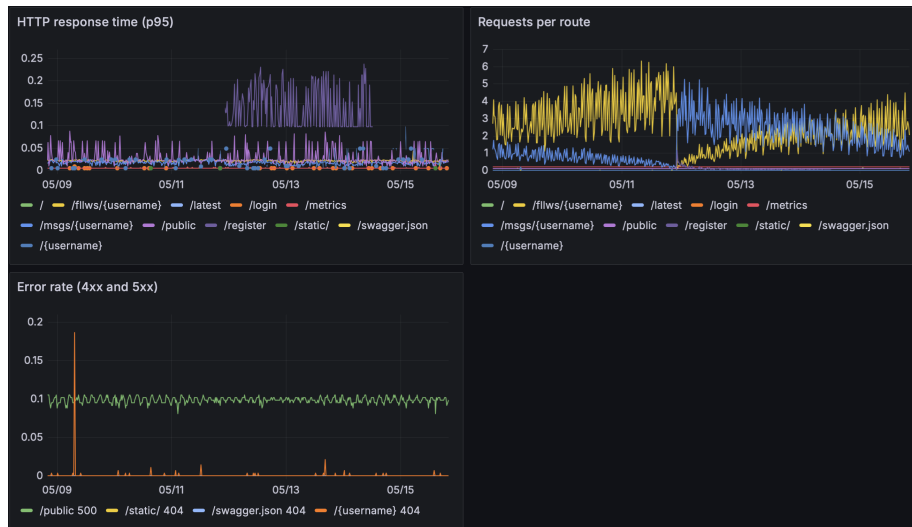


Figure 5: HTTP metrics: P95 response time per route, requests per route, and error rate

2.3 Logging

Author(s): Peter J

For logging we went with the Grafana LGTM stack — Promtail collects logs from every container and ships them to Loki, and Grafana lets us search and explore them.

Promtail runs as a global Swarm service, meaning one instance per node, so no replica gets skipped. It watches the Docker socket and auto-discovers containers, pulling in logs from all of them. For each container it tags logs with three labels: the container name, the Swarm service name (like `minitwit_webserver`), and the specific task/replica. These labels make it easy to filter down to exactly what you want in Grafana’s log explorer.

On the Loki side, we use a simple single-node setup with filesystem storage and daily index rotation. Logs older than 7 days are dropped at ingestion to keep disk usage in check. The app itself just writes to stdout via Go’s standard `log` package, which Docker captures automatically — no extra log library needed. We also enabled Traefik access logs so request-level data from the reverse proxy flows into Loki through the same pipeline, which turned out to be really useful for debugging.

The logging dashboard below shows log volume per Swarm service over time. Day-to-day it stays low and stable, with the occasional spike from Traefik or Loki itself when there is a burst of activity.



Figure 6: Log volume by Swarm service

2.4 Security Hardening

Author(s): Peter J

Multi-stage Docker build. The app Dockerfile uses two stages: a builder stage that compiles the Go binary, and a final `alpine:3.23` image that only contains the compiled binary, templates, and static files. The build tools are thrown away entirely, so the attack surface of the running container is minimal. We also create a dedicated non-root user (`appuser`) inside the image and drop to that user before the process starts, so even if something goes wrong inside the container it doesn't have root.

Firewall. Every node gets UFW rules applied automatically during Terraform provisioning. The manager node opens ports 80, 443, and 8080 for Traefik, plus SSH and the three Docker Swarm internal ports. Worker nodes only have the Swarm ports and SSH open — no public ports at all, so all traffic must go through Traefik on the manager. The database node only exposes SSH and Postgres (5432). On top of that, DigitalOcean's cloud firewall adds another layer at the network edge.

TLS. Traefik handles HTTPS via Let's Encrypt automatically. All HTTP traffic on port 80 is redirected to HTTPS, and certificates are stored in a named Docker volume so they persist across redeployments.

Secrets. Sensitive values like the database URL, app secret key, Discord webhook, and GHCR credentials are kept as Docker Swarm secrets or GitHub Actions secrets — never hardcoded or baked into images.

Static analysis. The CI pipeline runs `golangci-lint` on Go code, `hadolint` on the Dockerfile, and `Semgrep` for SAST on every push. These run as parallel jobs and block merges if they fail.

2.5 Availability and Scaling

Author(s): Peter J, Apoorva

The webserver runs as three replicas with a `spread: node.id` placement preference in Docker Swarm, which tries to put replicas on different nodes so a single node going down doesn't take the whole service with it. Traefik automatically distributes traffic across whichever replicas are healthy.

Rolling updates are set to deploy one replica at a time with a 10-second pause between steps and `order: start-first` — this means the new replica comes

up and passes its health check before the old one is taken down, so there is no gap in availability during a deploy. If an update goes bad, the rollback config mirrors the same approach, replacing one replica at a time.

Each replica runs a health check every 30 seconds (`wget --spider` against `localhost`) and Swarm stops routing to a replica if it starts failing. Combined with the Prometheus `WebserverDown` alert, we get two independent signals if something goes wrong — the load balancer stops sending traffic and we get a Discord notification within a minute.

The monitoring services (Prometheus, Grafana, Loki) each run as a single replica pinned to the manager node. We could have run them in a more resilient configuration, but these services all hold persistent state that is genuinely tricky to replicate without extra tooling, and a brief monitoring outage is much less bad than a complex distributed setup breaking in production. The webserver replicas are stateless and are the only part of the system we actually scale horizontally. If we ever needed more capacity on the database or monitoring side, vertical scaling (resizing the droplet) is the more practical option.

As mentioned above, the database tier follows a different scaling model from the web tier. The managed PostgreSQL instance on DigitalOcean serves as a vertically scalable service, so CPU, memory, or disk pressure is handled by upgrading the managed database rather than introducing our own multi-node database topology. That keeps the operational surface smaller while still matching simulator load.

We follow the same split across the project. We scaled our webserver horizontally in Swarm, and kept persistence and alerts easy to control by keeping the database and monitoring services centralized on the manager node.

3 Reflection Perspective

3.1 Evolution and Refactoring

Author(s): Håkon and Leo

The storyboard below puts the whole project on one page: course topics across the top, three lanes for what shipped on time vs >2 weeks late, and operational incidents underneath. We refer back to it from the later Reflection sections.

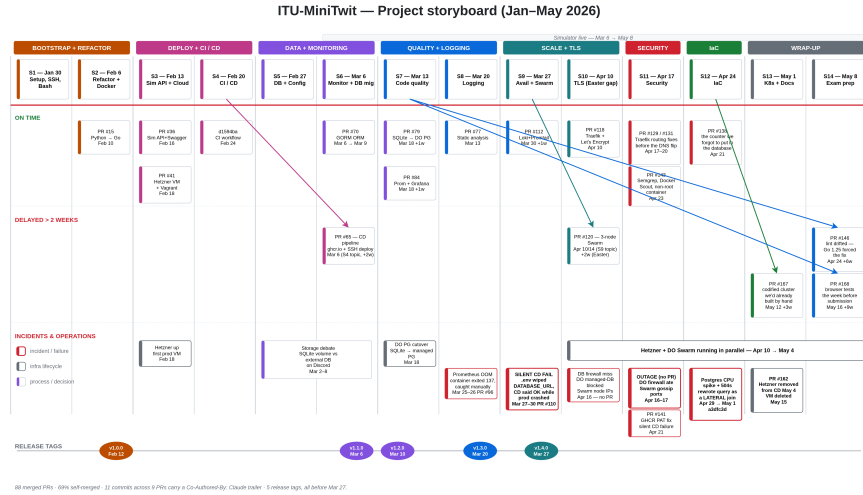


Figure 7: Project storyboard: thematic arcs, on-time vs delayed PRs, and operational incidents from Jan to May 2026

The project had two main architectural rewrites following the course outline: an early Python-to-Go port (PR #15, week 2) and the later move from a single Hetzner deployment to a three-node Docker Swarm cluster on DigitalOcean (PR #120 and follow-ups). The Swarm migration was the more consequential as it changed the system from one server running everything to replicated webserver, Traefik routing, managed PostgreSQL, and Swarm secrets. Running three webserver replicas also forced us to move shared state out of memory. One example was the `latest` simulator counter, which we moved into PostgreSQL (PR #138).

The same pattern appeared elsewhere. The personal timeline query seemed fine in early testing, but later timed out for users with many follows. It took several rounds of diagnosis across the team before we landed on the query rewrite that fixed it (`a3dfc3d`).

In hindsight, we mostly refactored when something forced us to. That kept the project moving, but it also meant that some weaknesses only became visible when the system was under pressure.

3.2 Operation

Author(s): Leo and Apoorva

As mentioned, we ran two production environments in parallel for most of April: a single-node Hetzner deployment and a Docker Swarm cluster on DigitalOcean,

both connected to the same managed PostgreSQL instance. This let us migrate gradually without interrupting the simulator.

One sharp operational lesson came from that parallel-run window. On 16 April, a new DigitalOcean firewall blocked Docker Swarm’s internal overlay traffic between nodes. The manager kept serving traffic through its local replica, so external uptime checks stayed green while cluster redundancy had silently failed for nearly 18 hours. Later Swarm routing follow-ups on DigitalOcean (PR #129 / #131) addressed a separate set of overlay-routing bugs in the lead-up to the DNS migration.

The incident showed that control-plane and data-plane failures are different things. Edge-level uptime checks were not enough. Three replicas behind Traefik gave us horizontal replication, but we never benchmarked whether it was better than the simpler single-node setup. The manager also remained a single point of failure for several critical services.

The project’s operational work also tied monitoring back to deployment quality and observability: in the later stages, CI reliability was improved around Codacy and Playwright, and follow-up work around managed PostgreSQL alerts and Grafana credential handling made the stack easier to operate after deploys.

The alerting setup also reflects an operational tradeoff: the DigitalOcean database alerts were kept sensitive enough to catch CPU, memory, and disk pressure early, but not so aggressive that they would interrupt ongoing work with false alarms.

Setting up the Grafana credentials highlighted a key operational constraint: mounting a persistent volume is not enough to preserve the admin password across redeloys, because the password is still governed by container startup configuration. That meant a redeploy from **master** could overwrite the password even when persistent storage remained intact.

Additionally, to verify the functionality of our CI/CD pipelines, whenever the workflow YAML changed significantly, the pipeline was triggered manually to verify jobs such as Playwright and Codacy, and to confirm that Docker Scout still failed only on critical and high vulnerabilities while lower-severity findings could continue to be observed during development.

3.3 Maintenance

Author(s): Leo

Our maintenance work was mostly reactive. Tooling improved during the project, with linting, security scanning, and image hardening added over time rather than from the start. A concrete example was Codacy, which identified a `/health` route bug that had been live for weeks (fixed in commit `c8ff76c`, whose message reads *“caught by codacy”*).

The storyboard above shows the same pattern in the DELAYED lane: maintenance tooling often arrived only after problems became visible. PR #146

updated the lint setup when it drifted out of sync with newer Go versions, and PR #168 added browser tests in the week before submission. More generally, bugs that affected visible behaviour were fixed, while quieter problems remained. For example, bcrypt errors are still swallowed in helper code, simulator authentication contains hardcoded values, test coverage is limited, and logs accumulated recurring warnings that nobody investigated.

The lesson is that maintenance needs an owner. Improvements happened when a problem became painful enough to fix, not because we had a regular practice for improving maintainability.

3.4 DevOps Style

Author(s): Leo

Using the DevOps Handbook’s Three Ways as a lens, flow was actually our weakest area. We had CI/CD and PRs from early on (PR #65), but we never set up a project board, issue tracking, an estimation practice, or a Kanban view. Work was visible only through Discord pings and the PR queue. Batches were often too large — the 69% self-merge rate shown in the storyboard reflects PRs that grew too big and too sole-authored to be reviewed.

Feedback was mixed. Prometheus and Loki were up by the end, but for most of the project we read server health by SSH-ing into the containers manually rather than through dashboards. When things broke we did swarm on Discord and people grabbed tasks fast — that informal feedback loop worked, even if the tooling-driven one didn’t. The 16 April firewall outage is the clearest evidence the formal loop was incomplete: edge uptime stayed green for 18 hours.

Continual learning was probably our strongest area, but informally so. We met collaboratively each week to understand work done over the weekends, drew architecture on the whiteboard together, and briefed each other on Discord. The larger incidents got written up (`docs/incidents/session11-ops-debug.md`, `docs/architecture/latest-counter-in-db.md`), and the `docs/` folder is structured into `incidents/`, `operations/`, `architecture/`, `security/`, and `monitoring/` — but we never established a regular post-mortem practice or allocated dedicated improvement time. In hindsight, two sessions a week instead of one would have helped.

The main takeaway is that DevOps was not just about adding tools. It became real when we had to operate the system ourselves.

4 Use of Generative AI

Author(s): Leo

We used Anthropic Claude, mainly through the Code interface, throughout the project. AI-assisted commits carry `Co-Authored-By: Claude`, and `.mailmap`

maps the tool to LLM `<none>` as required by the course. Across merged PRs, the trailer appears on 11 commits in 9 PRs (shown in the storyboard footer above). The real number is higher: squash-merging strips trailers (only 5 survive on master), and much of the AI help during debugging, exploration, and prose never made it into a tagged commit.

The clearest place AI helped was the early refactor of the inherited Python/Flask app into Go (PR #15). None of us knew Go, and Claude helped us scaffold the package structure and understand compiler errors while we learned the type system. Later, the same kind of help was useful for Docker Swarm, Traefik, the PostgreSQL migration, CI/CD setup, and security tooling.

It only helped when we could evaluate the suggestions. Plausible but wrong answers sometimes slowed debugging down, so we learned to treat AI as a fast helper for exploration, not as an authority.

AI use was also uneven across the team. It increased individual productivity, but it also meant that some contributors could move faster through unfamiliar technical areas than others. Overall, generative AI helped us move faster, but only when paired with our own understanding.